



Texas Hold'em Poker Game

A Full-Stack Python + JavaScript Web Application

Computer Science Project



What Did We Build?



Real Poker Game

Full Texas Hold'em rules — blinds, betting rounds, hand evaluation



Python Backend

Flask API manages game state, player actions & hand logic



JavaScript Frontend

Browser-based UI — no frameworks, pure HTML/CSS/JS



Docker Deployed

Containerized with Docker Compose, runs anywhere



Backend — Python & Flask

Key Files

<code>main.py</code>	Starts the Flask server
<code>game.py</code>	Core game loop & rules
<code>game_session.py</code>	Manages active game state
<code>player.py</code>	Player model & actions
<code>table.py</code>	Table, seats, pot logic
<code>evaluator.py</code>	Hand strength comparison
<code>cards.py</code>	Deck, shuffle, deal

How It Works

- 1 Browser sends action (fold, call, raise...)
- 2 Flask API receives the request
- 3 Game logic updates state (cards, pot, turn)
- 4 Evaluator checks who wins the hand
- 5 JSON response is sent back to browser



app.js — Frontend Game Logic

app.js — Poker_game

Update

0.0.0.0:8000 JS app.js

EXPLORER

- POKER_GAME
 - __pycache__
 - backend
 - static
 - JS app.js
 - index.html
 - style.css
 - .dockerignore
 - cards.py
 - Dockerfile
 - evaluator.py
 - game_session.py
 - main.py
 - player.py
 - requirements.txt
 - table.py
 - workstations
 - .gitignore
 - cards.py
 - docker-compose.yml
 - evaluator.py
 - game.py
 - helpful_terminology.png
 - LICENSE
 - main.py
 - player.py
 - README.md
 - table.py
 - ui.py
- OUTLINE
- TIMELINE

```
backend > static > JS app.js > doAction
1  /* Texas Hold'em frontend */
2
3  let gameId = null;
4
5  // — Card rendering —
6
7  const RANK_LABEL = { T: "10", J: "J", Q: "Q", K: "K", A: "A" };
8  const SUIT_SYMBOL = { H: "♥", D: "♦", C: "♣", S: "♠" };
9
10 function renderCard(code) {
11   if (code === "??") {
12     return `<div class="card card-back" title="Hidden"></div>`;
13   }
14
15   const rankChar = code[0];
16   const suitChar = code[1];
17   const rank = RANK_LABEL[rankChar] ?? rankChar;
18   const suit = SUIT_SYMBOL[suitChar] ?? suitChar;
19   const color = (suitChar === "H" || suitChar === "D") ? "red" :
20
21   return `
22     <div class="card ${color}" title="${rank}${suit}">
23       <div class="card-top"><span class="card-rank">${rank}</span>
24       <div class="card-center">${suit}</div>
25       <div class="card-bottom"><span class="card-rank">${rank}</s
26     </div>`;
27 }
28
29 function renderCards(codes) {
30   if (!codes || codes.length === 0) {
31     return `<span class="no-cards-msg"></span>`;
32   }
33   return codes.map(renderCard).join("");
34 }
35
36 // — State rendering —
37
38 function render(state) {
```



Frontend — HTML / CSS / JS



index.html

The main page structure — table, cards, player seats, action buttons



style.css

Green felt theme, card styles, animations — dark & gold color palette



app.js

Sends player actions to API, updates the UI when state changes



style.css — Visual Design

EXPLORER

POKER_GAME

__pycache__

backend

static

JS app.js

index.html

style.css

.dockerignore

cards.py

Dockerfile

evaluator.py

game_session.py

main.py

player.py

requirements.txt

table.py

workstations

.gitignore

cards.py

docker-compose.yml

evaluator.py

game.py

helpful_terminology.png

LICENSE

main.py

player.py

README.md

table.py

ui.py

OUTLINE

TIMELINE

0.0.0.0:8000

style.css

backend > static > # style.css > :root

```
1 /* — Reset & base
2 *, *::before, *::after { box-sizing: border-box; margin: 0; paddi
3
4 :root {
5   --felt: #1a6b3a;
6   --felt-dark: #145530;
7   --felt-rim: #0d3d22;
8   --bg: #0f0f1a;
9   --gold: #f0c040;
10  --card-red: #cc1111;
11  --card-black: #111111;
12  --log-bg: #12121f;
13  --text: #e8e8e8;
14  --muted: #888;
15  --radius: 10px;
16 }
17
18 body {
19   background: var(--bg);
20   color: var(--text);
21   font-family: 'Segoe UI', system-ui, sans-serif;
22   min-height: 100vh;
23 }
```

Ports

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

(base) → Poker_game git:(main) docker compose up --build

1.1" 200 OK

poker-1 | INFO: 192.168.65.1:18492 - "GET /api/logs/stream HTT

P/1.1" 200 OK

poker-1 | INFO: 192.168.65.1:45574 - "GET /api/logs/stream HTT

P/1.1" 200 OK

poker-1 | INFO: 192.168.65.1:37324 - "GET /api/logs/stream HTT

P/1.1" 200 OK

poker-1 | INFO: 192.168.65.1:28453 - "GET /api/logs/stream HTT

P/1.1" 200 OK

poker-1 | INFO: 192.168.65.1:31497 - "GET /api/logs/stream HTT

P/1.1" 200 OK

□

View in Docker Desktop

View Config

View Logs

Enable W

main

0 0 0

Screen Reader Optimized

Ln 15, Col 22

Spaces: 2

UTF-8

LF

{ } CSS



How a Game Round Works

Deal

Pre-Flop

Flop

Turn

River

Showdown

Each player gets 2 cards

First betting round

3 community cards
revealed

4th community card

5th community card

Best hand wins the pot

Player Actions Each Round:

Fold

Give up your hand

Check

Pass (no bet needed)

Call

Match the current bet

Raise

Increase the bet



Full Project Structure

Frontend (Browser)

- index.html

- style.css

- app.js

Backend (Flask / Python)

- main.py → Flask server

- game.py → rules

- evaluator.py → winner

Infrastructure

- Docker container

- docker-compose.yml

- requirements.txt

Browser ↔ Flask API ↔ Docker



What We Built — Summary



Full Texas Hold'em game playable in any browser



Python backend handles all game logic & rules



Clean JavaScript frontend — no heavy frameworks



Dockerized — easy to run anywhere with one command



Modular code — separate files for game, players, cards



Thank you! Any questions?

